

Time Series Analysis

Lukas Arnold	Simone Arnold	Florian Bagemihl
Matthias Baitsch	Marc Fehr	Maik Poetzsch
	Sebastian Seipel	

2026-09-17

Table of contents

Preamble	4
Introduction	5
1 1 Classification: Time Series as a Measurement Process	6
2 1 Classification: Time Series as a Measurement Process	7
3 2 Preprocessing & Validation of a Time Series	12
4 2 Preprocessing & Validation of a Time Series	13
5 3 Resampling & Alignment	18
6 3 Resampling & Alignment	19
7 4 Smoothing & Noise Reduction	25
8 4 Smoothing & Noise Reduction	26
9 5 Trend & Season	34
10 5 Trend & Season	35
11 6 Lags & Autocorrelation	42
12 6 Lags & Autocorrelation	43
13 7 Anomalies	49
14 7 Anomalies	50
15 8 Forecasting Baselines & Evaluation	57
16 8 Forecasting Baselines & Evaluation	58
17 9 Learning Objectives Check	65

18 9 Learning Objectives Check	66
19 Exercise 10: Time Series Workflow	68
20 Exercise 10: Time Series Workflow	69
21 11 Exam Questions	71
22 11 Exam Questions	72

Preamble



Building Blocks of Computer-Based Data Analysis. “Time Series Analysis” by Lukas Arnold, Simone Arnold, Florian Bagemihl, Matthias Baitsch, Marc Fehr, Maik Poetzsch, and Sebastian Seipel is licensed under [CC BY 4.0](#). The work is available at <https://github.com/bausteine-der-datenanalyse/m-analyse-von-zeitdaten-en>. Excluded from the license are all logos and other specially marked content. 2024

Suggested Citation

Arnold, Lukas, Simone Arnold, Matthias Baitsch, Marc Fehr, Maik Poetzsch, and Sebastian Seipel. 2024. “Building Blocks of Computer-Based Data Analysis. Module: Time Series Analysis.” <https://github.com/bausteine-der-datenanalyse/m-analyse-von-zeitdaten-en>.

BibTeX Template

```
@misc{BCD-Styleguide-2024,  
  title={Building Blocks of Computer-Aided Data Analysis. Time Series Analysis Module},  
  author={Arnold, Lukas and Arnold, Simone and Baitsch, Matthias and Fehr, Marc and Poetzsch,  
  year={2024},  
  url={https://github.com/bausteine-der-datenanalyse/m-analyse-von-zeitdaten-  
en}  
}
```

Introduction

Prerequisites

- Python Basics
- NumPy (Arrays, basic operations)
- Matplotlib (Visualization)
- Basic understanding of Pandas (Series, DataFrame)

Packages and Datasets Used

Packages

- [NumPy](#)
- [Matplotlib](#)
- [Pandas](#)

Datasets

Estimated Time

Estimated completion time: 2 hours

Learning Objectives

- Preprocess time series in a structured way (time index, missing values, outliers)
- Apply resampling and alignment methodically and correctly
- Choose smoothing techniques appropriate for the question at hand
- Identify and interpret trend and seasonal components
- Diagnose temporal dependencies using lag/autocorrelation
- Detect anomalies using simple, explainable rules
- Implement forecast baselines and evaluate them with error metrics

1 1 Classification: Time Series as a Measurement Process

2 1 Classification: Time Series as a Measurement Process

A time series is the result of a **measurement process over time**.

Typical examples:

- Sensor data
- Energy consumption
- Web traffic
- Financial data
- Machine data in production processes

What matters is not only the numerical value itself, but also:

- **Sampling rate** (hourly, minute-level, irregular?)
- **Measurement quality**
- **Structural patterns** (trend, periodicity)
- **Disturbances** (noise, outliers)

Remember

Time series analysis does not start with a model, but with an understanding of the measurement process.

2.1 1.1 Creating a Working Dataset

We create a synthetic time series with:

- Trend
- Daily periodicity
- Noise
- Outliers

This structure is typical for real-world technical measurement data.

2.2 Pandas (recommended)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)

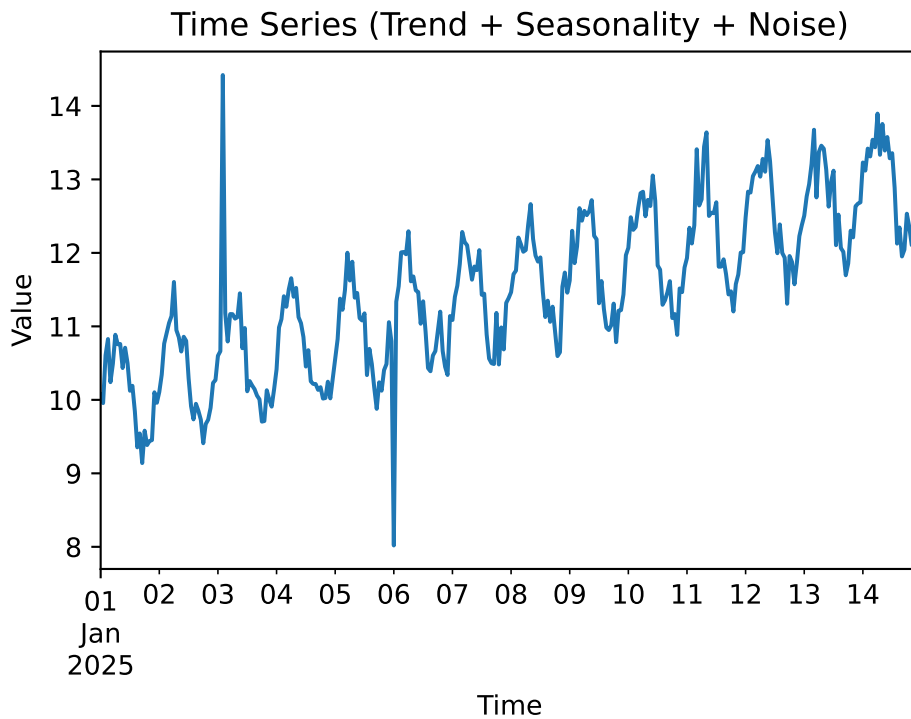
# Time index: 14 days, hourly measurements
t = pd.date_range("2025-01-01", periods=24*14, freq="h")

trend = np.linspace(0, 3, len(t))
season = 0.8 * np.sin(2*np.pi*(t.hour)/24)
noise = 0.25 * rng.normal(size=len(t))

y = 10 + trend + season + noise
y = pd.Series(y, index=t, name="signal")

# artificial outliers
y.iloc[50] += 3.5
y.iloc[120] -= 2.8

y.plot()
plt.title("Time Series (Trend + Seasonality + Noise)")
plt.xlabel("Time")
plt.ylabel("Value")
plt.show()
```



2.3 NumPy (Principle)

Here we model time as a simple sample index.

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)

n = 24*14
x = np.arange(n)

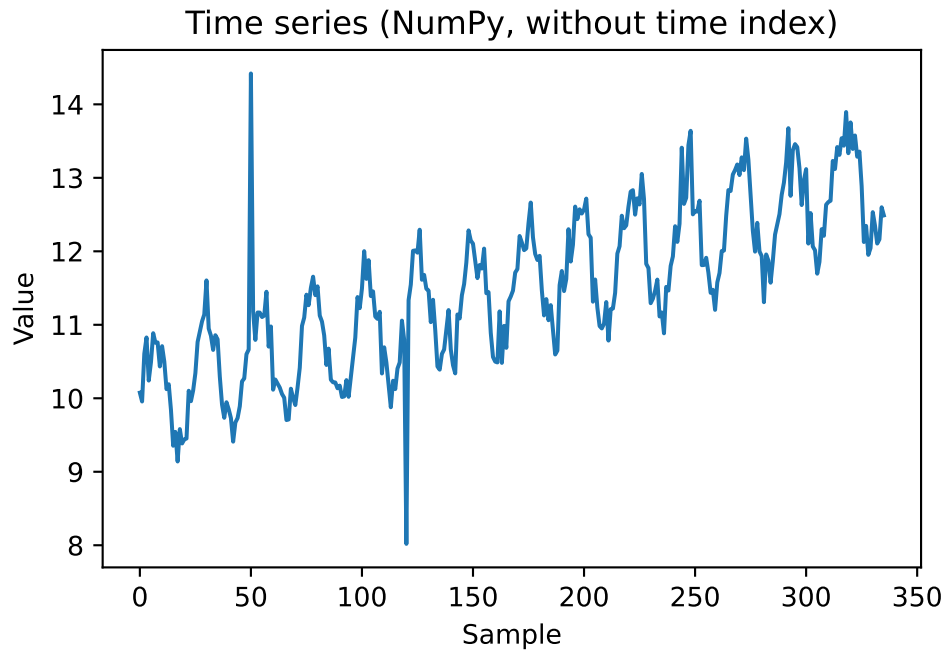
trend = np.linspace(0, 3, n)
season = 0.8 * np.sin(2*np.pi*(x % 24)/24)
noise = 0.25 * rng.normal(size=n)

y = 10 + trend + season + noise

# Outliers
```

```
y[50] += 3.5
y[120] -= 2.8

plt.plot(y)
plt.title("Time series (NumPy, without time index)")
plt.xlabel("Sample")
plt.ylabel("Value")
plt.show()
```



2.4 1.2 Typical Analysis Questions

In time series analysis, similar questions arise repeatedly:

- Is there a **trend**?
- Does a **periodicity** exist (e.g., daily or weekly patterns)?
- How strong is the **noise**?
- Are there **anomalies**?
- How can future values be roughly estimated?

Methodological Perspective

Before smoothing or modeling:

1. Visualize
2. Check the time scale
3. Identify the structure
4. Only then continue with further analysis

2.5 1.3 Structural Model of a Time Series

A time series can often be conceptually represented as:

Signal = Trend + Seasonality + Noise + Outliers

This decomposition is not a physical law, but rather a **conceptual model** that helps analyze patterns systematically.

Attention

Not every recurring pattern is automatically “seasonality”.
Regularity must plausibly match the sampling rate and the underlying process.

2.6 1.4 Mini Reflection

Answer the following:

1. Which periodicity was modeled here?
2. What role does the sampling rate play?
3. Would daily aggregation make the seasonal component visible or invisible?
4. How would outliers behave under strong smoothing?

These questions will accompany us throughout the entire methods module.

3 2 Preprocessing & Validation of a Time Series

4 2 Preprocessing & Validation of a Time Series

Before a time series is analyzed, it must be **formally correct and consistent**.

Many errors in later analyses arise from:

- unsorted timestamps
- duplicate timestamps
- missing values
- irregular sampling
- implicit time gaps

Remember

Time series analysis is only as good as the underlying data. Validation is not an optional step.

4.1 2.1 Example Time Series with Problems

We intentionally create a faulty time series (unsorted, duplicates, missing values).

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(1)

t = pd.date_range("2025-01-01", periods=24*3, freq="h")
y = pd.Series(10 + rng.normal(size=len(t)), index=t, name="signal")

# create problems
y = y.sample(frac=1)                                # unsorted
y = pd.concat([y, y.iloc[5:8]])                      # duplicates (append is deprecated/removed)
y.iloc[10:15] = np.nan                             # missing values
```

```
y.head()
```

```
2025-01-01 04:00:00    10.905356
2025-01-03 19:00:00    10.759520
2025-01-03 01:00:00    10.891167
2025-01-02 09:00:00    12.042772
2025-01-03 23:00:00     9.702473
Name: signal, dtype: float64
```

4.2 2.2 Step 1: Ensure Sorting

```
y = y.sort_index()
```

4.3 2.3 Step 2: Detecting and Handling Duplicates

Detect duplicates:

```
dup_mask = y.index.duplicated(keep=False)
y[dup_mask].head(10)
```

```
2025-01-01 08:00:00    10.364572
2025-01-01 08:00:00    10.364572
2025-01-01 09:00:00    10.294132
2025-01-01 09:00:00    10.294132
2025-01-02 11:00:00    10.663063
2025-01-02 11:00:00    10.663063
Name: signal, dtype: float64
```

Duplicates must be handled methodically, for example:

- Average per timestamp (typical for measurements)
- Keep first/last value (typical for status updates)
- Remove (if clearly erroneous)

Example: Average per timestamp

```
y = y.groupby(level=0).mean()
```

4.4 2.4 Step 3: Analyze Missing Values

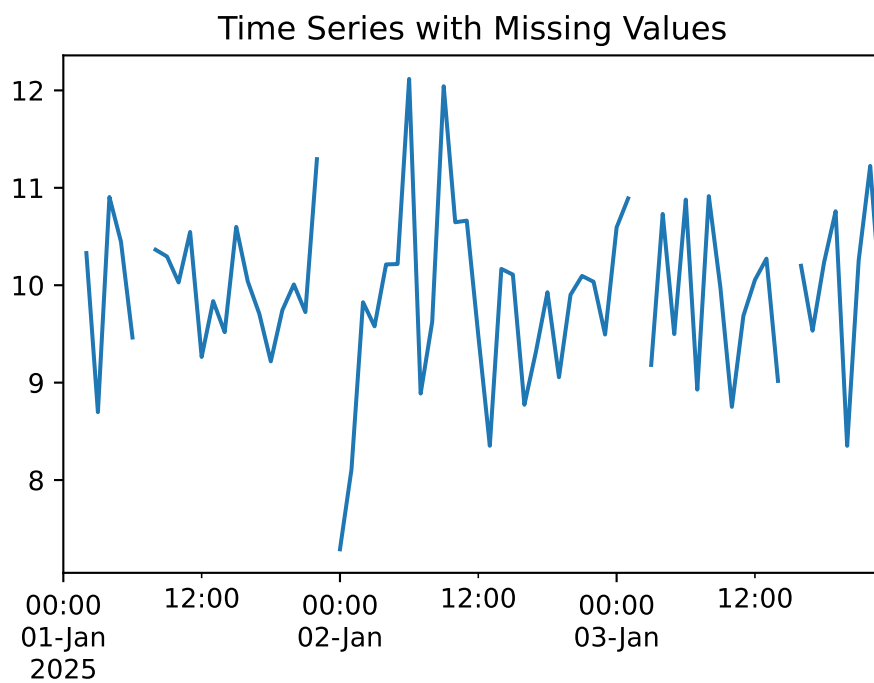
```
y.isna().sum()
```

```
np.int64(5)
```

4.4.1 Visualizing the Gaps

```
import matplotlib.pyplot as plt

y.plot(title="Time Series with Missing Values")
plt.show()
```



4.4.2 Treatment options

```
y_ffill = y.ffmpeg()
y_interp = y.interpolate("time")
```



Decision Helper

- `ffill`: useful for state data (“applies until the next measurement”)
- `interpolate("time")`: useful for continuous measurements

4.5 2.5 Step 4: Check Sampling Rate

Is the time series regular?

```
diffs = y.index.to_series().diff()
diffs.value_counts().head()
```

```
0 days 01:00:00    71
Name: count, dtype: int64
```

If multiple time intervals occur → irregular sampling or time gaps.

4.6 2.6 Making Time Gaps Explicit

Using `asfreq` enforces the expected frequency. Missing time points appear as NaN.

```
y_regular = y.asfreq("h")
y_regular.isna().sum()
```

```
np.int64(5)
```



Warning

Without an explicit frequency, gaps in time may remain invisible.

4.7 2.7 Validation Checklist

Before proceeding with further analysis:

- ☐ Index is a `DatetimeIndex`
- ☐ Time series is sorted
- ☐ Duplicates identified and properly handled
- ☐ Missing values identified and a strategy chosen
- ☐ Sampling rate checked
- ☐ Frequency (if applicable) explicitly set

4.8 2.8 Mini-Exercise

Create your own time series with:

- irregular sampling
- 5 missing timestamps
- 3 duplicates

Then implement a proper validation according to the checklist.

5 3 Resampling & Alignment

6 3 Resampling & Alignment

Time series often come at different sampling rates or need to be transformed to a different time scale.

Common reasons:

- Comparing two time series
- Reducing noise through aggregation
- Adjusting to reporting intervals
- Preparing for modeling

i Note

Resampling means changing the time scale. You must decide between aggregation and interpolation.

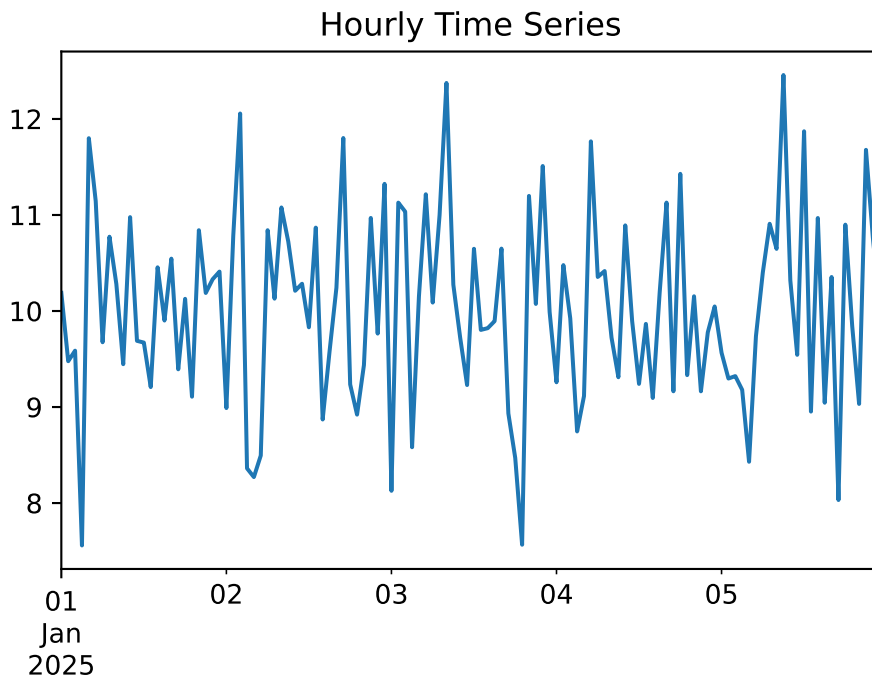
6.1 3.1 Original Time Series (Hourly)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(2)

t = pd.date_range("2025-01-01", periods=24*5, freq="h")
y = pd.Series(10 + rng.normal(size=len(t)), index=t, name="signal")

y.plot(title="Hourly Time Series")
plt.show()
```

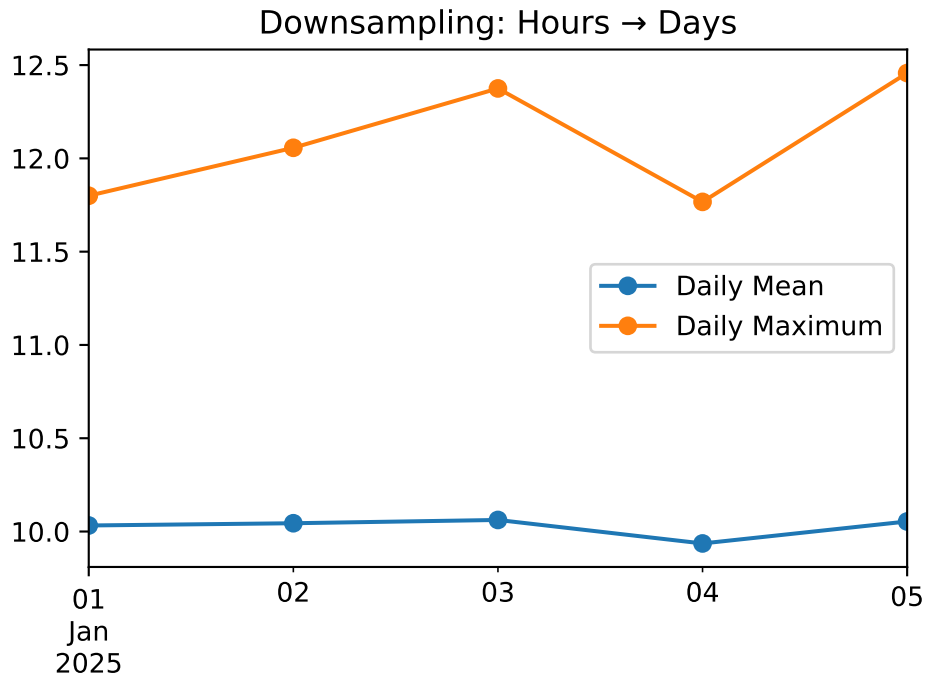


6.2 3.2 Downsampling (Aggregation)

We aggregate from hours to days.

```
daily_mean = y.resample("D").mean()
daily_max = y.resample("D").max()

ax = daily_mean.plot(marker="o", label="Daily Mean")
daily_max.plot(ax=ax, marker="o", label="Daily Maximum")
plt.legend()
plt.title("Downsampling: Hours → Days")
plt.show()
```



💡 Decision Guide

- Mean → smooths out fluctuations
- Maximum → highlights extreme values
- Sum → useful for quantities (e.g., energy consumption)

6.3 3.3 Upsampling (Interpolation)

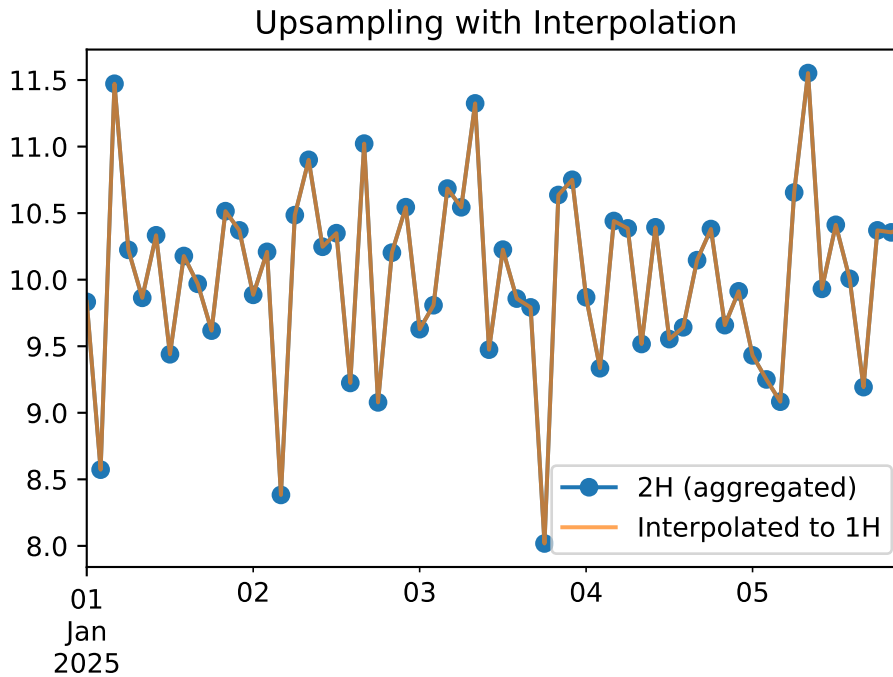
We increase the temporal resolution.

```
y_2h = y.resample("2h").mean()  
y_up = y_2h.resample("h").interpolate("time")
```

```

ax = y_2h.plot(marker="o", label="2H (aggregated)")
y_up.plot(ax=ax, alpha=0.7, label="Interpolated to 1H")
plt.legend()
plt.title("Upsampling with Interpolation")
plt.show()

```



⚠ Warning

Interpolation does not generate new measurement data; it only models a plausible progression.

6.4 3.4 Alignment of Two Time Series

Two time series with different sampling rates:

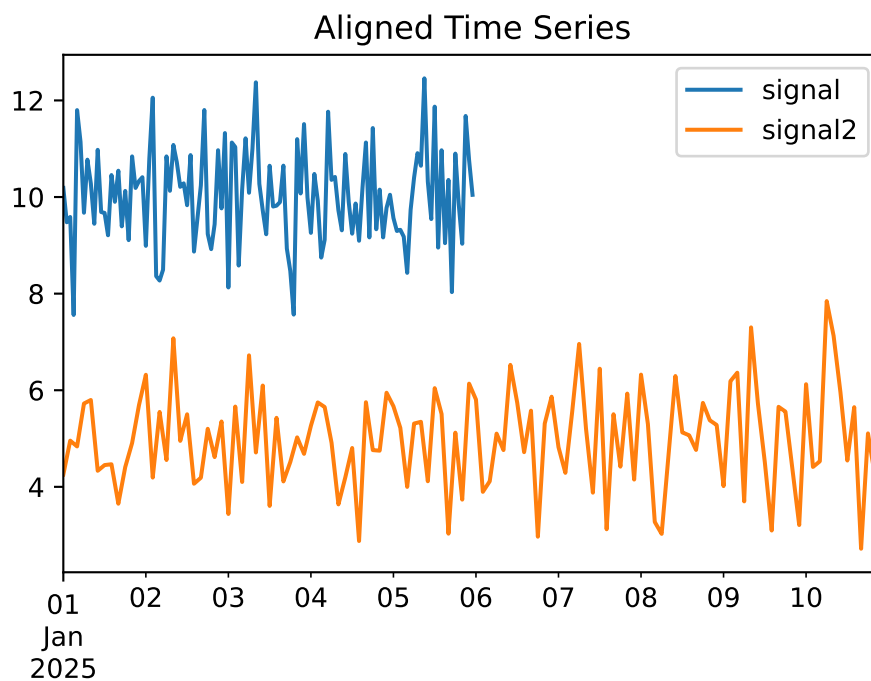
```
t2 = pd.date_range("2025-01-01", periods=24*5, freq="2h")
y2 = pd.Series(5 + rng.normal(size=len(t2)), index=t2, name="signal2")
```

For joint analysis, both series must have the same frequency.

```
y2_aligned = y2.resample("h").interpolate("time")

df = pd.concat([y, y2_aligned], axis=1)

df.plot(title="Aligned Time Series")
plt.show()
```



i Note

Alignment means: - same frequency - same time span - same time zone

6.5 3.5 Common Sources of Error

- Comparisons without prior resampling
 - Mixing local time and UTC
 - Aggregation without considering data types (e.g., sum vs mean)
-

6.6 3.6 Mini Exercise

1. Generate a minute-level time series over 2 days.
2. Aggregate to hourly averages.
3. Compare visually:
 - Original
 - Hourly averages
4. Interpret the loss of information.

7 4 Smoothing & Noise Reduction

8 4 Smoothing & Noise Reduction

Smoothing is one of the most important standard techniques in time series analysis. It helps to:

- reduce noise,
- make trends/seasonality more visible,
- define a **baseline** (e.g., for anomalies).

Note

The window width is a methodological choice:
it should match the **time scale of the question** being investigated.

8.1 4.1 Example Signal

We use a time series with a daily pattern, noise, and individual outliers.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)

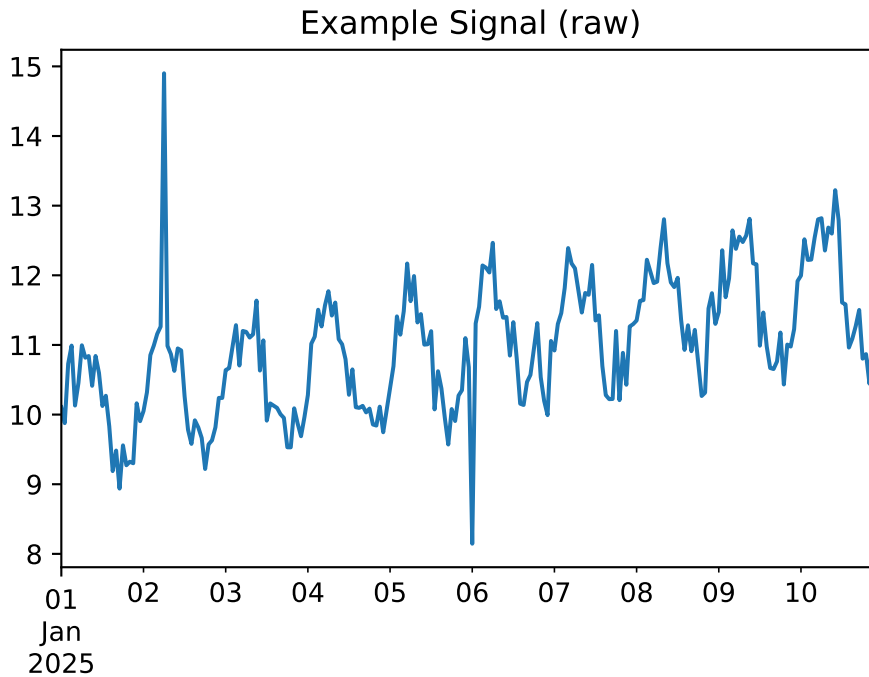
t = pd.date_range("2025-01-01", periods=24*10, freq="h") # 10 days hourly
trend = np.linspace(0, 2, len(t))
season = 0.9 * np.sin(2*np.pi*(t.hour)/24)
noise = 0.35 * rng.normal(size=len(t))

s = pd.Series(10 + trend + season + noise, index=t, name="signal")

# Outliers
s.iloc[30] += 3.0
```

```
s.iloc[120] -= 2.5

s.plot(title="Example Signal (raw)")
plt.show()
```



8.2 4.2 Rolling Mean

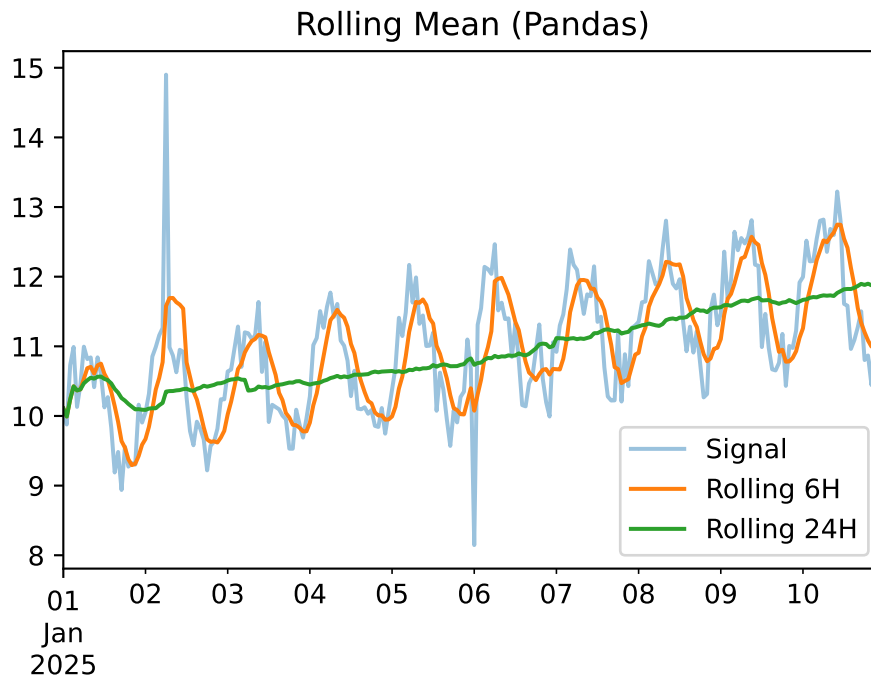
The rolling mean is the classic smoothing method: the average over a window.

8.3 Pandas (recommended)

```
roll_6h = s.rolling("6h").mean()
roll_24h = s.rolling("24h").mean()

ax = s.plot(alpha=0.45, label="Signal")
roll_6h.plot(ax=ax, label="Rolling 6H")
roll_24h.plot(ax=ax, label="Rolling 24H")
plt.legend()
```

```
plt.title("Rolling Mean (Pandas)")
plt.show()
```



💡 Decision aid

- **6H**: reduces short-term noise, events remain visible
- **24H**: shows basic level/trend, strongly dampens daily patterns

8.4 NumPy (principle)

NumPy variant: moving average as convolution (`convolve`).

```
import numpy as np
import matplotlib.pyplot as plt

y = s.to_numpy()

def moving_average(y, window):
    kernel = np.ones(window) / window
```

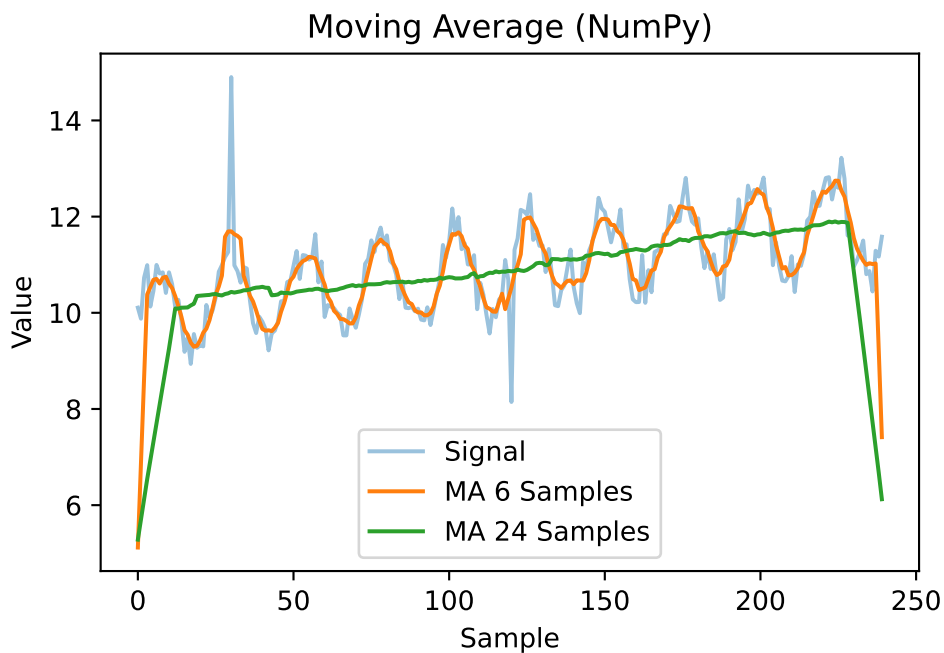
```

    return np.convolve(y, kernel, mode="same")

ma_6 = moving_average(y, 6)
ma_24 = moving_average(y, 24)

plt.plot(y, alpha=0.45, label="Signal")
plt.plot(ma_6, label="MA 6 Samples")
plt.plot(ma_24, label="MA 24 Samples")
plt.legend()
plt.title("Moving Average (NumPy)")
plt.xlabel("Sample")
plt.ylabel("Value")
plt.show()

```



⚠ Attention

When convolving, you must consciously accept edge handling (here: `mode="same"`). Pandas handles edge areas consistently with `min_periods` and NaNs.

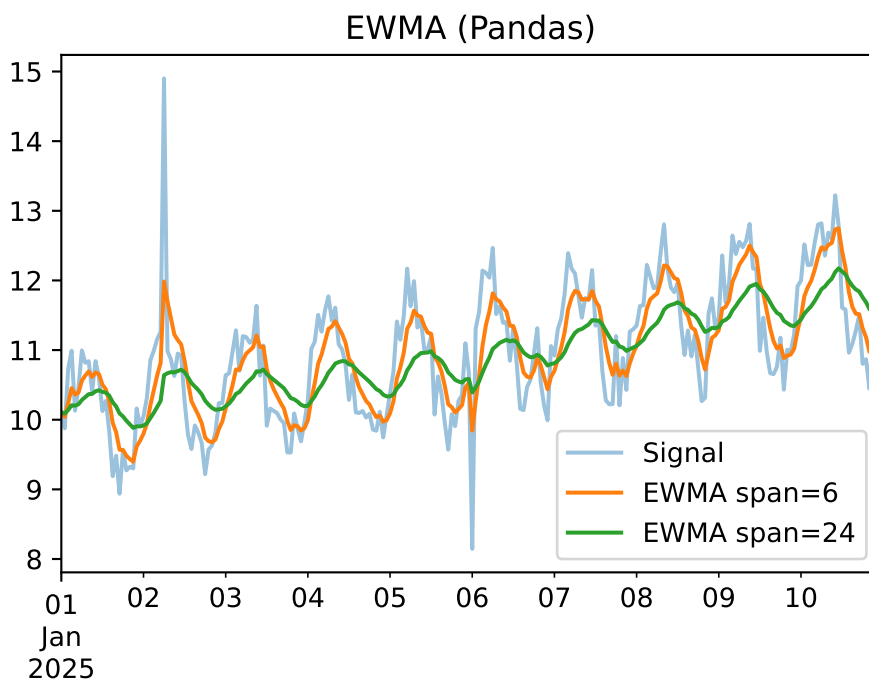
8.5 4.3 EWMA (Exponentially Weighted Moving Average)

EWMA smooths, but weights newer values more heavily.

8.6 Pandas (recommended)

```
ewm_fast = s.ewm(span=6, adjust=False).mean()
ewm_slow = s.ewm(span=24, adjust=False).mean()

ax = s.plot(alpha=0.45, label="Signal")
ewm_fast.plot(ax=ax, label="EWMA span=6")
ewm_slow.plot(ax=ax, label="EWMA span=24")
plt.legend()
plt.title("EWMA (Pandas)")
plt.show()
```



i Note

EWMA reacts faster to changes than a rolling mean, because older values have exponentially less influence.

8.7 NumPy (Concept)

EWMA can be calculated recursively:

$$m_t = \alpha \cdot y_t + (1 - \alpha) \cdot m_{t-1}$$

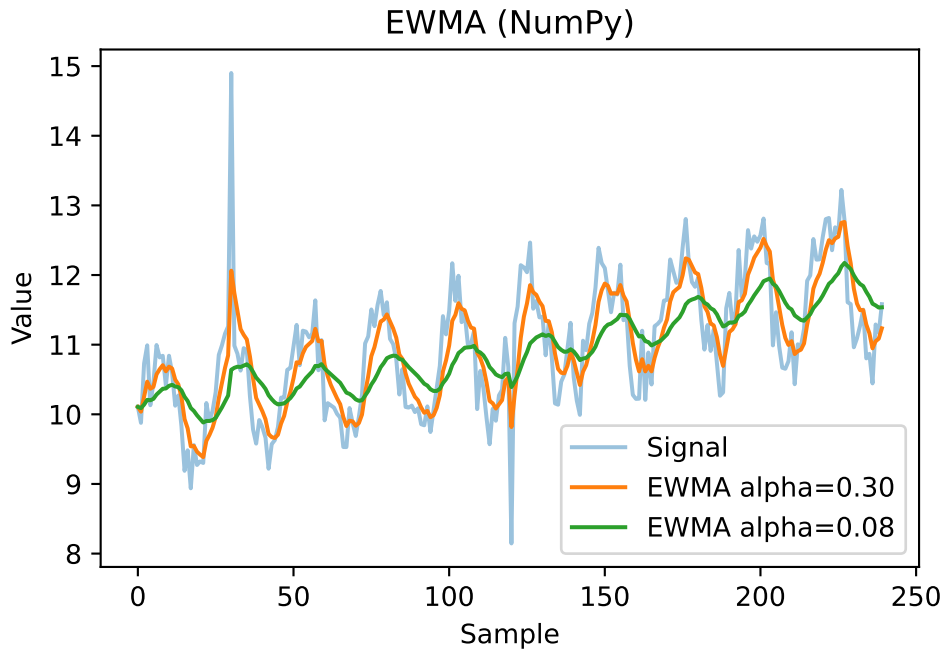
```
import numpy as np
import matplotlib.pyplot as plt

y = s.to_numpy()

def ewma(y, alpha):
    m = np.empty_like(y, dtype=float)
    m[0] = y[0]
    for i in range(1, len(y)):
        m[i] = alpha * y[i] + (1 - alpha) * m[i-1]
    return m

m_fast = ewma(y, alpha=0.30)
m_slow = ewma(y, alpha=0.08)

plt.plot(y, alpha=0.45, label="Signal")
plt.plot(m_fast, label="EWMA alpha=0.30")
plt.plot(m_slow, label="EWMA alpha=0.08")
plt.legend()
plt.title("EWMA (NumPy)")
plt.xlabel("Sample")
plt.ylabel("Value")
plt.show()
```



8.8 4.4 Method Choice: Which Approach When?

💡 Decision Aid

Rolling Mean is suitable when...

- You want to average over a clearly defined time window (e.g., “last 24 hours”).

EWMA is suitable when...

- You want a smooth baseline that reacts more quickly to changes.
- You want to detect anomalies as deviations from a dynamic baseline.

8.9 4.5 Mini-Exercises

1. **Window Selection:** Test rolling windows of 3H, 12H, 48H.
 - Which structures become visible or disappear in each case?
2. **Anomalies & Smoothing:** Increase the outlier amplitude.
 - At what smoothing strength does the outlier “disappear” in the smoothed signal?
3. **EWMA Sensitivity:** Vary `span` (Pandas) or `alpha` (NumPy).

- Which parameters respond “quickly” and which “slowly”? Explain why.

9 5 Trend & Season

10 5 Trend & Season

Many time series contain systematic structures:

- **Trend:** long-term development (increasing/decreasing)
- **Season:** periodically recurring pattern
- **Residual:** remaining fluctuation

This decomposition is a **conceptual model** that helps to systematically analyze structure.

Note

Trend describes a slow change,
Season describes a regular repetition.

10.1 5.1 Example Signal

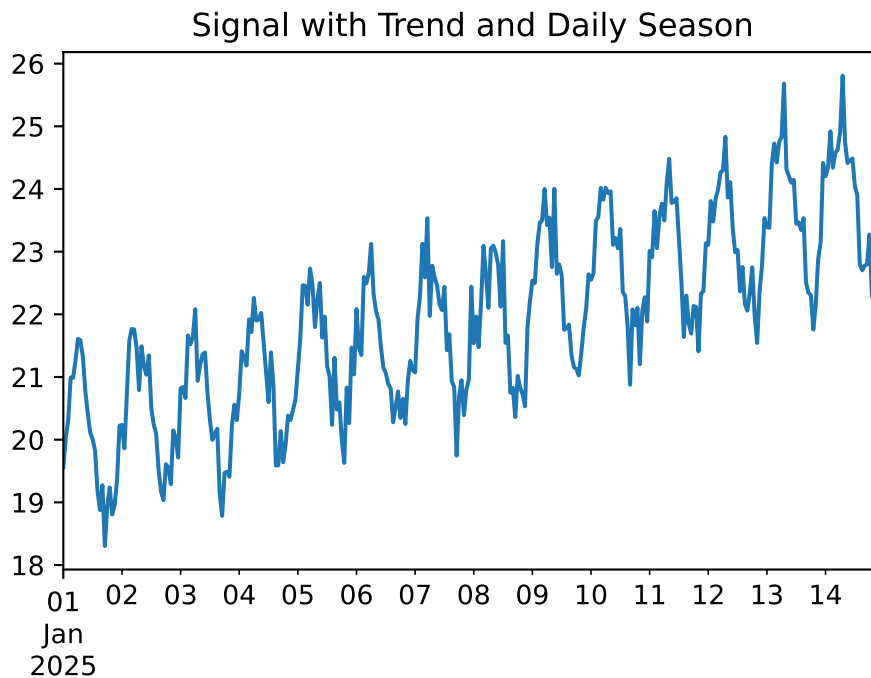
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(10)

t = pd.date_range("2025-01-01", periods=24*14, freq="h")
trend = np.linspace(0, 4, len(t))
season = 1.2 * np.sin(2*np.pi*(t.hour)/24)
noise = 0.4 * rng.normal(size=len(t))

s = pd.Series(20 + trend + season + noise, index=t, name="signal")
```

```
s.plot(title="Signal with Trend and Daily Season")
plt.show()
```

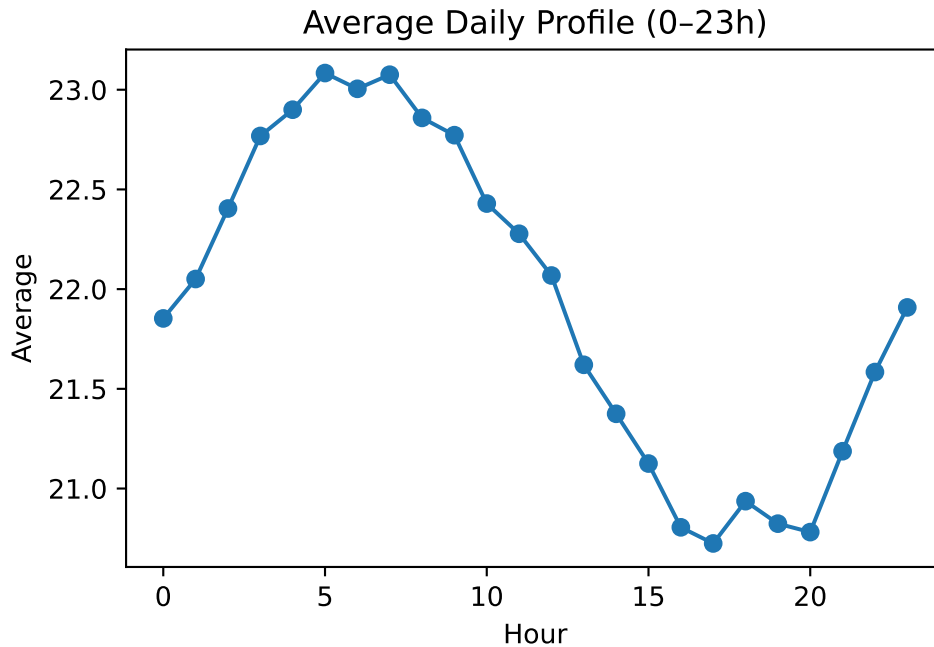


10.2 5.2 Visualizing the Season

10.2.1 Hourly Average

```
season_hour = s.groupby(s.index.hour).mean()

season_hour.plot(marker="o")
plt.title("Average Daily Profile (0-23h)")
plt.xlabel("Hour")
plt.ylabel("Average")
plt.show()
```



💡 Interpretation

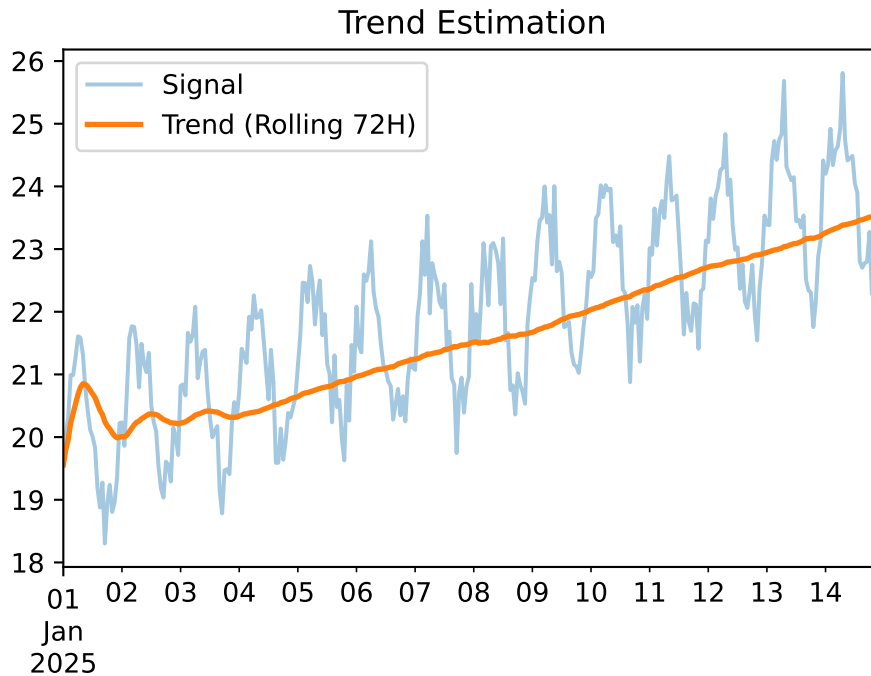
A periodic pattern in this visualization suggests a daily seasonality.

10.3 5.3 Estimating the Trend (Strong Smoothing)

10.4 Pandas

```
trend_est = s.rolling("72h").mean()

ax = s.plot(alpha=0.4, label="Signal")
trend_est.plot(ax=ax, linewidth=2, label="Trend (Rolling 72H)")
plt.legend()
plt.title("Trend Estimation")
plt.show()
```



10.5 NumPy (Concept)

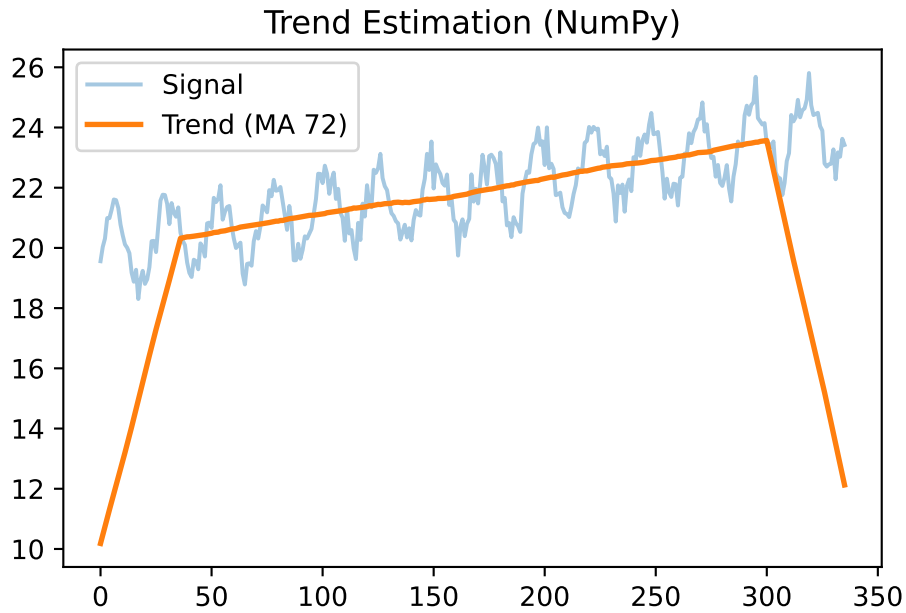
```
import numpy as np

y = s.to_numpy()

def moving_average(y, window):
    kernel = np.ones(window) / window
    return np.convolve(y, kernel, mode="same")

trend_np = moving_average(y, 72)

plt.plot(y, alpha=0.4, label="Signal")
plt.plot(trend_np, linewidth=2, label="Trend (MA 72)")
plt.legend()
plt.title("Trend Estimation (NumPy)")
plt.show()
```

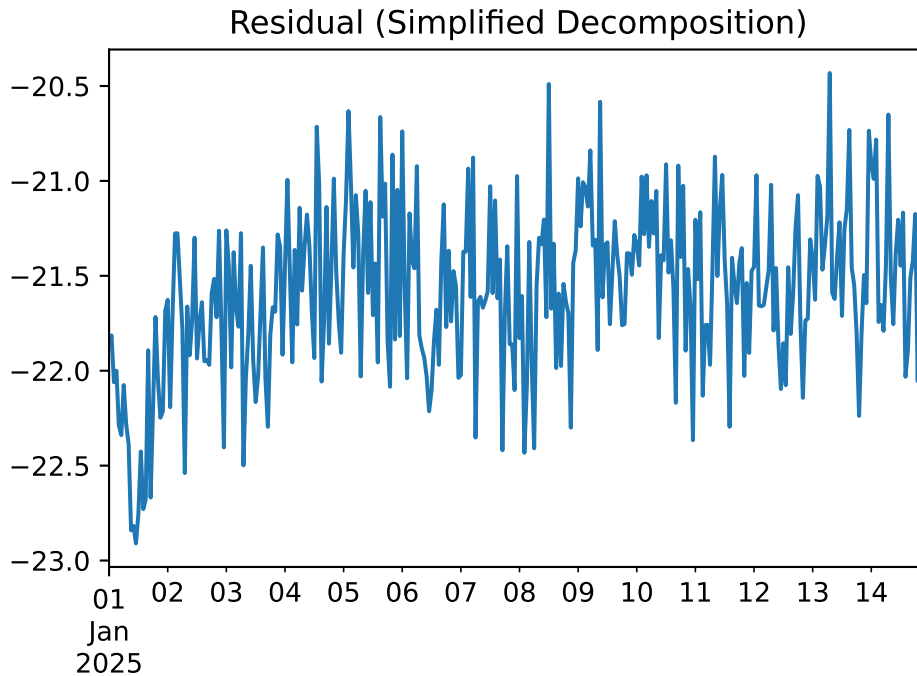


10.6 5.4 Calculating the Residual (Simplified Decomposition)

We roughly remove the trend and seasonality:

```
season_map = s.index.hour.map(season_hour)
residual = s - trend_est - season_map

residual.plot(title="Residual (Simplified Decomposition)")
plt.show()
```



⚠ Warning

This decomposition is didactically simplified.
In practice, more robust methods exist (e.g., STL).
Here, the focus is on the principle.

10.7 5.5 Additive vs Multiplicative Model

Additive model:

Signal = Trend + Season + Noise

Multiplicative model:

Signal = Trend × Season × Noise

Multiplicative models make sense
when the amplitude of the season grows with the level.

10.8 5.6 Mini-Exercises

1. Increase the trend slope.
 - How does this affect the detection of the season?
2. Generate a weekly season (period length 7 days).
 - How would you make this visible?
3. Test a very large rolling window (e.g., 120H).
 - Which structure remains?

11 6 Lags & Autocorrelation

12 6 Lags & Autocorrelation

Time series differ from classical datasets because observations are **not independent**.

The value at time t often depends on previous values.

We examine this temporal dependence using:

- **Lags**
- **Autocorrelation**

i Note

Temporal dependence is a core feature of time series. Without it, meaningful forecasting would not be possible.

12.1 6.1 Example Signal

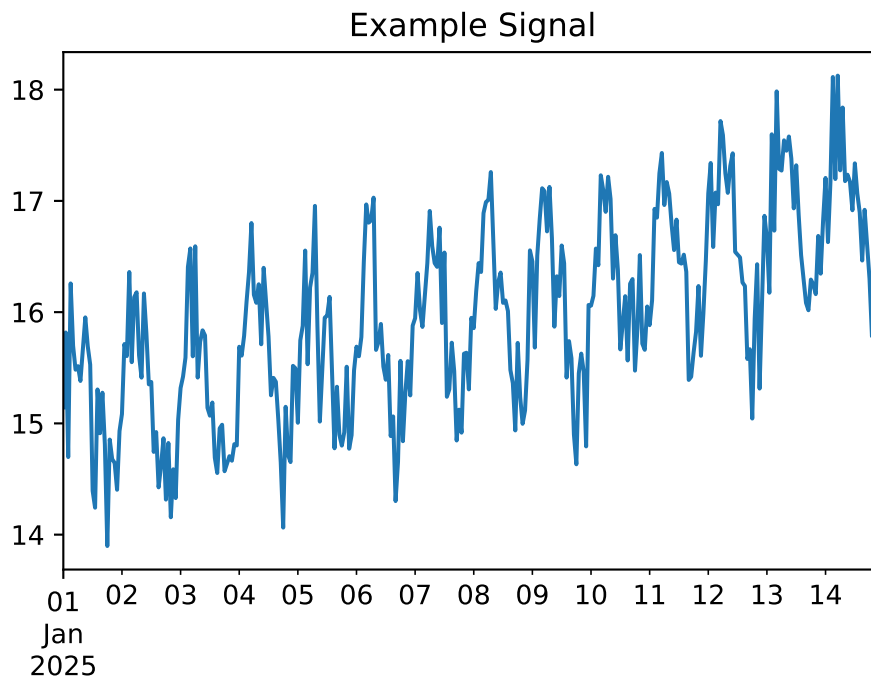
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(21)

t = pd.date_range("2025-01-01", periods=24*14, freq="h")
trend = np.linspace(0, 2, len(t))
season = 0.8 * np.sin(2*np.pi*(t.hour)/24)
noise = 0.4 * rng.normal(size=len(t))

s = pd.Series(15 + trend + season + noise, index=t, name="signal")
```

```
s.plot(title="Example Signal")
plt.show()
```



12.2 6.2 Lag Features

A lag shifts the time series backward.

12.3 Pandas

```
lag_1h = s.shift(1)
lag_24h = s.shift(24)

df_lags = pd.DataFrame({
    "y": s,
    "lag_1h": lag_1h,
```

```

    "lag_24h": lag_24h
})

df_lags.head()

```

	y	lag_1h	lag_24h
2025-01-01 00:00:00	15.143509	NaN	NaN
2025-01-01 01:00:00	15.817296	15.143509	NaN
2025-01-01 02:00:00	14.697408	15.817296	NaN
2025-01-01 03:00:00	16.258241	14.697408	NaN
2025-01-01 04:00:00	15.697774	16.258241	NaN

12.4 NumPy (Concept)

```

y = s.to_numpy()

lag1 = np.roll(y, 1)
lag24 = np.roll(y, 24)

lag1[0] = np.nan
lag24[:24] = np.nan

```

Interpretation

A high correlation between $y(t)$ and $y(t-24)$ indicates a daily periodicity.

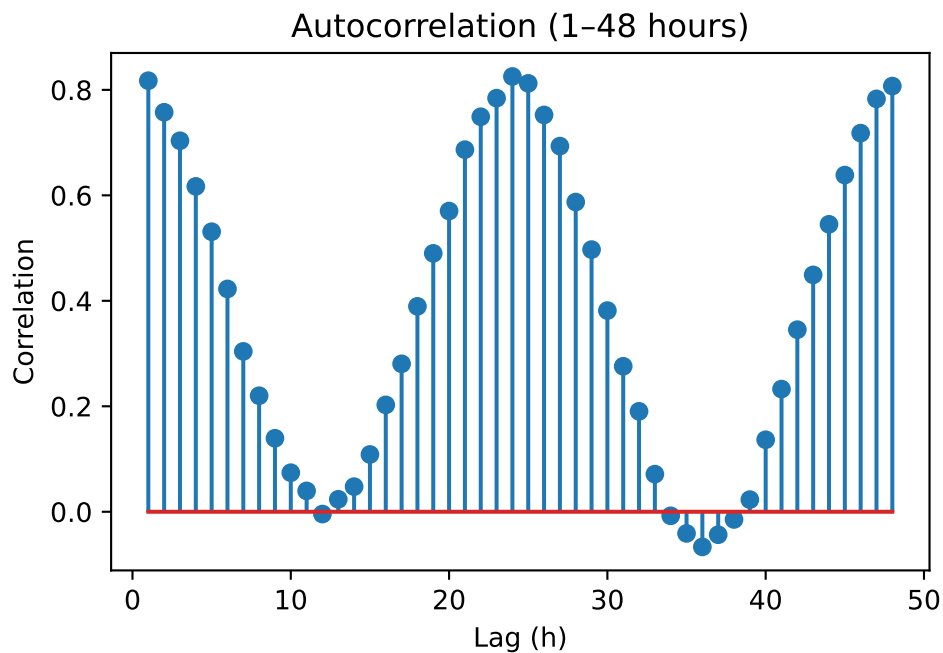
12.5 6.3 Calculating Autocorrelation

Autocorrelation measures the correlation between $y(t)$ and $y(t-k)$.

12.6 Pandas

```
lags = range(1, 49)
ac = [s.corr(s.shift(k)) for k in lags]

plt.stem(list(lags), ac)
plt.title("Autocorrelation (1-48 hours)")
plt.xlabel("Lag (h)")
plt.ylabel("Correlation")
plt.show()
```



12.7 NumPy (Principle)

```
import numpy as np
import matplotlib.pyplot as plt

y = s.to_numpy()
y = y - np.nanmean(y)

def autocorr(y, k):
```

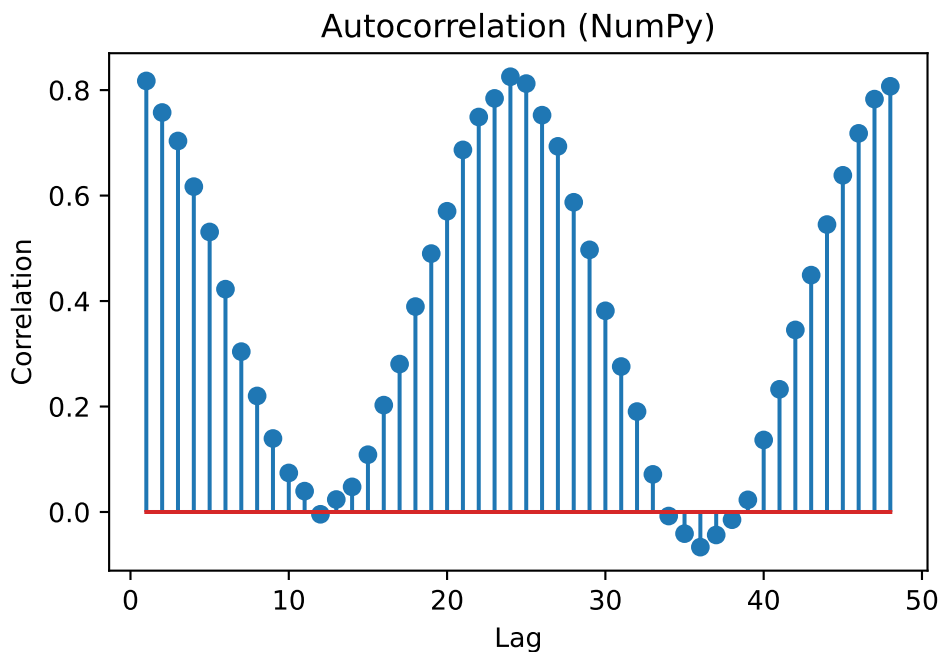
```

return np.corrcoef(y[:-k], y[k:])[0,1]

lags = range(1, 49)
ac_np = [autocorr(y, k) for k in lags]

plt.stem(list(lags), ac_np)
plt.title("Autocorrelation (NumPy)")
plt.xlabel("Lag")
plt.ylabel("Correlation")
plt.show()

```



12.8 6.4 Interpretation of Autocorrelation

Typical patterns:

- Slowly decreasing curve → strong trend
- Periodic peaks (e.g., at 24, 48, 72) → seasonality
- Rapid drop toward 0 → weak dependence

 Warning

A trend can artificially increase autocorrelation.
If necessary, first detrend or difference the series.

12.9 6.5 Mini Exercises

1. Remove the trend (e.g., using a rolling mean).
 - How does the autocorrelation change?
2. Create a time series without seasonality.
 - What does the autocorrelation function look like?
3. Generate purely random data.
 - What do you expect?

13 7 Anomalies

14 7 Anomalies

Anomalies are observations that deviate significantly from expected behavior.

In time series this means:

- strong deviation from the trend
- strong deviation from the seasonal pattern
- sudden structural breaks

Important: anomalies are **context-dependent**.

i Remember

An anomaly is not purely a statistical property.
It is a deviation from the expected behavior of a process.

14.1 7.1 Example Signal with Outliers

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

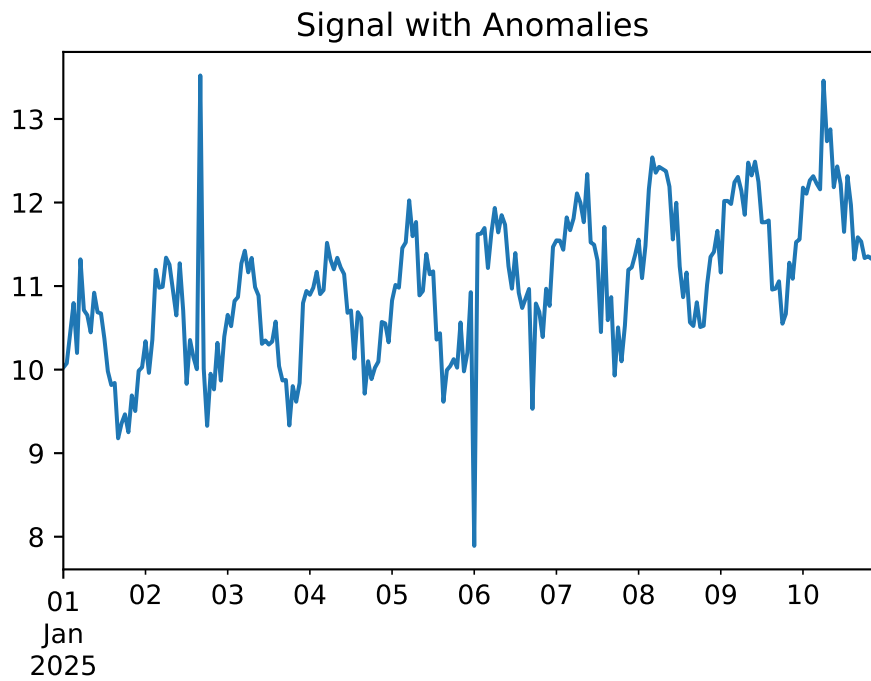
rng = np.random.default_rng(99)

t = pd.date_range("2025-01-01", periods=24*10, freq="h")
trend = np.linspace(0, 2, len(t))
season = 0.8 * np.sin(2*np.pi*(t.hour)/24)
noise = 0.3 * rng.normal(size=len(t))
```

```
s = pd.Series(10 + trend + season + noise, index=t, name="signal")

# artificial anomalies
s.iloc[40] += 4
s.iloc[120] -= 3

s.plot(title="Signal with Anomalies")
plt.show()
```



14.2 7.2 Global Z-Score Method

We standardize the entire time series.

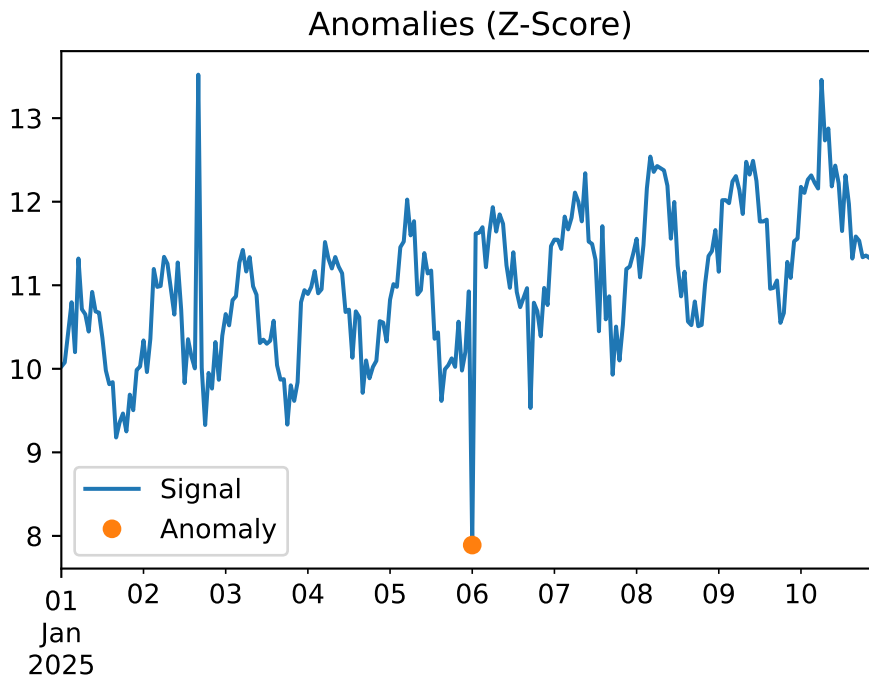
14.3 Pandas

```

z = (s - s.mean()) / s.std()
anomalies = s[abs(z) > 3]

ax = s.plot(label="Signal")
anomalies.plot(ax=ax, linestyle="None", marker="o", label="Anomaly")
plt.legend()
plt.title("Anomalies (Z-Score)")
plt.show()

```



14.4 NumPy

```

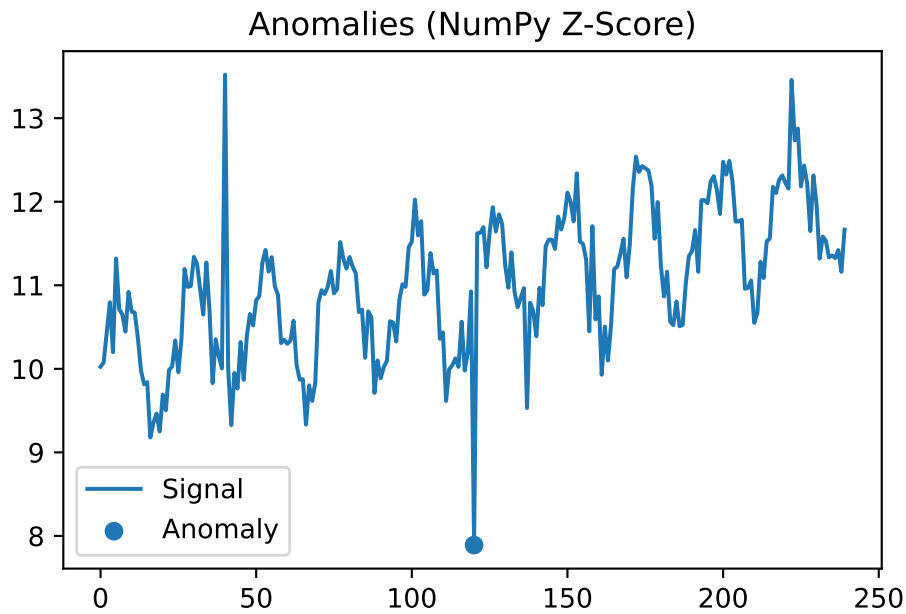
y = s.to_numpy()

z_np = (y - np.mean(y)) / np.std(y)
anomalies_idx = np.where(np.abs(z_np) > 3)[0]

plt.plot(y, label="Signal")
plt.scatter(anomalies_idx, y[anomalies_idx], label="Anomaly")
plt.legend()

```

```
plt.title("Anomalies (NumPy Z-Score)")
plt.show()
```



⚠ Warning

Z-score performs poorly when there is a strong trend or seasonality, because global means can become distorted.

14.5 7.3 Local Anomaly (Baseline + Residual)

Instead of global standardization, we define a local baseline.

```
baseline = s.ewm(span=24, adjust=False).mean()
residual = s - baseline

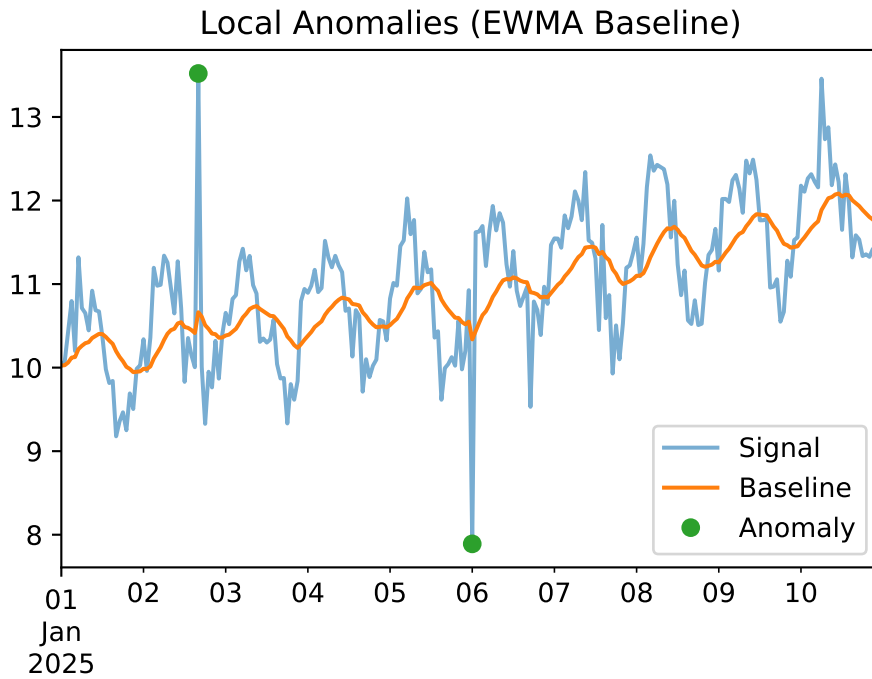
score = (residual - residual.mean()) / residual.std()
```

```

anomalies_local = s[abs(score) > 3]

ax = s.plot(alpha=0.6, label="Signal")
baseline.plot(ax=ax, label="Baseline")
anomalies_local.plot(ax=ax, linestyle="None", marker="o", label="Anomaly")
plt.legend()
plt.title("Local Anomalies (EWMA Baseline)")
plt.show()

```



💡 Interpretation

Local methods are more robust when a trend or seasonality is present.

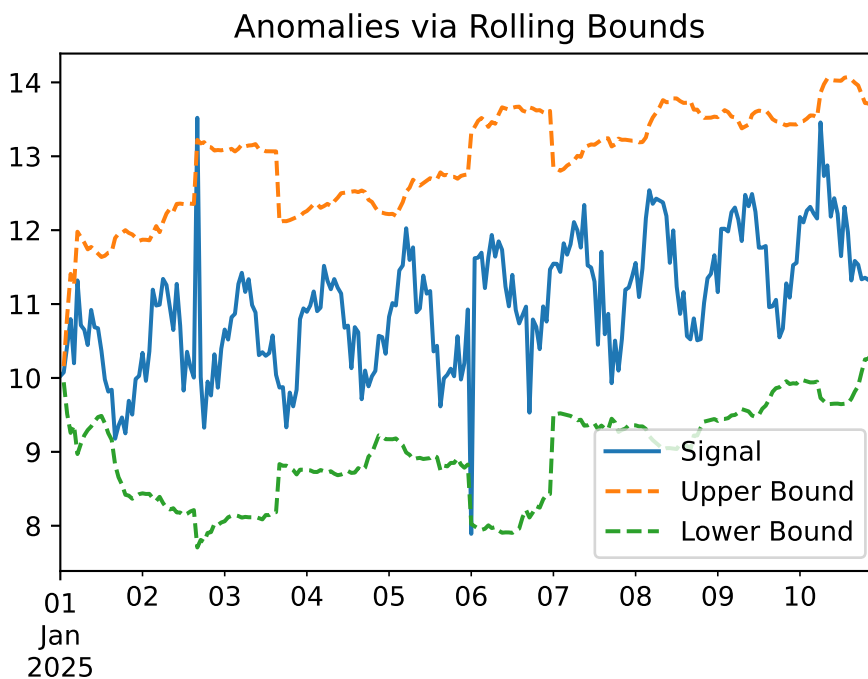
14.6 7.4 Rolling Standard Deviation

Anomalies can also be defined as strong deviations relative to the local variance.

```
rolling_std = s.rolling("24h").std()
rolling_mean = s.rolling("24h").mean()

upper = rolling_mean + 3 * rolling_std
lower = rolling_mean - 3 * rolling_std

ax = s.plot(label="Signal")
upper.plot(ax=ax, linestyle="--", label="Upper Bound")
lower.plot(ax=ax, linestyle="--", label="Lower Bound")
plt.legend()
plt.title("Anomalies via Rolling Bounds")
plt.show()
```



14.7 7.5 Structural Break vs Outlier

A single peak $\rightarrow$ Outlier

A permanent level shift $\rightarrow$ Structural Break

Both require different interpretation.

i Methodological Note

Before marking anomalies:

1. Check the trend
2. Check seasonality
3. Define the baseline
4. Justify the threshold

14.8 7.6 Mini-Exercises

1. Remove the trend before applying the Z-score.
 - How does the result change?
2. Lower the threshold from 3 to 2.
 - What happens?
3. Simulate a permanent level jump.
 - Is it detected as an anomaly?

15 8 Forecasting Baselines & Evaluation

16 8 Forecasting Baselines & Evaluation

Before applying complex models, simple **baseline models** should always be defined.

Baselines answer the question:

Is my model truly better than a simple, understandable strategy?

i Note

A complex model only makes sense if it clearly outperforms robust baselines.

16.1 8.1 Example Time Series

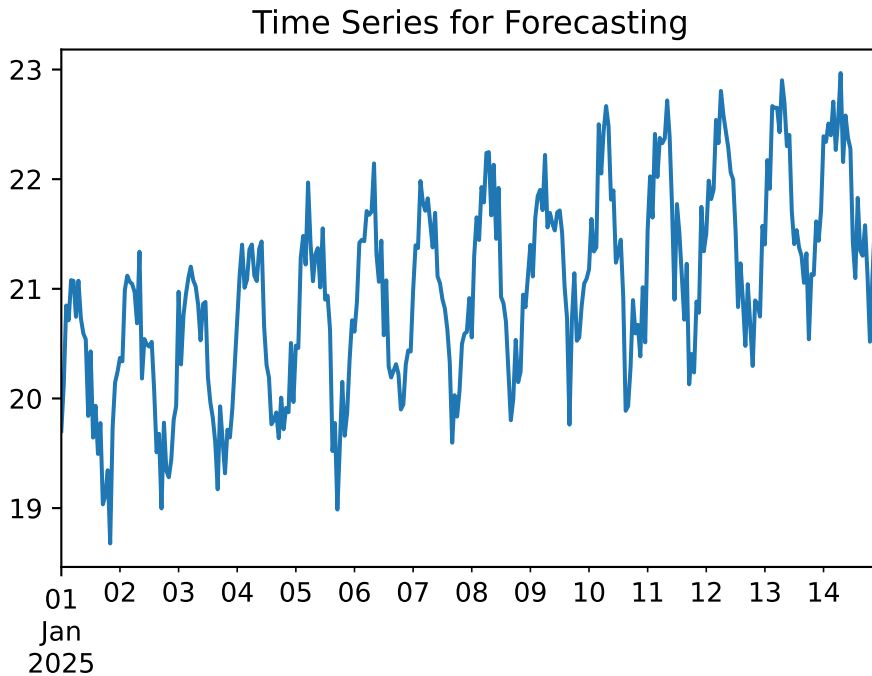
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(123)

t = pd.date_range("2025-01-01", periods=24*14, freq="h")
trend = np.linspace(0, 2, len(t))
season = 0.9 * np.sin(2*np.pi*(t.hour)/24)
noise = 0.3 * rng.normal(size=len(t))

s = pd.Series(20 + trend + season + noise, index=t, name="signal")

s.plot(title="Time Series for Forecasting")
plt.show()
```



16.2 8.2 Train-Test Split

We use 80% of the data for training and 20% for testing.

```
split = int(len(s) * 0.8)

train = s.iloc[:split]
test = s.iloc[split:]

train.index.min(), train.index.max(), test.index.min(), test.index.max()
```

```
(Timestamp('2025-01-01 00:00:00'),
 Timestamp('2025-01-12 03:00:00'),
 Timestamp('2025-01-12 04:00:00'),
 Timestamp('2025-01-14 23:00:00'))
```

16.3 8.3 Naive Forecast

Next value = last observed value.

16.4 Pandas

```
naive = test.copy()
naive[:] = train.iloc[-1]
```

16.5 NumPy

```
y = s.to_numpy()
train_np = y[:split]
test_np = y[split:]

naive_np = np.full_like(test_np, train_np[-1])
```

16.6 8.4 Seasonal Naive (24 Hours)

Next value = value from 24 hours ago.

16.7 Pandas

```
seasonal_naive = s.shift(24).iloc[split:]
```

16.8 NumPy

```
seasonal_naive_np = y[split-24:-24]
```

16.9 8.5 Moving Average Baseline

Forecast = mean of the last 24 hours.

16.10 Pandas

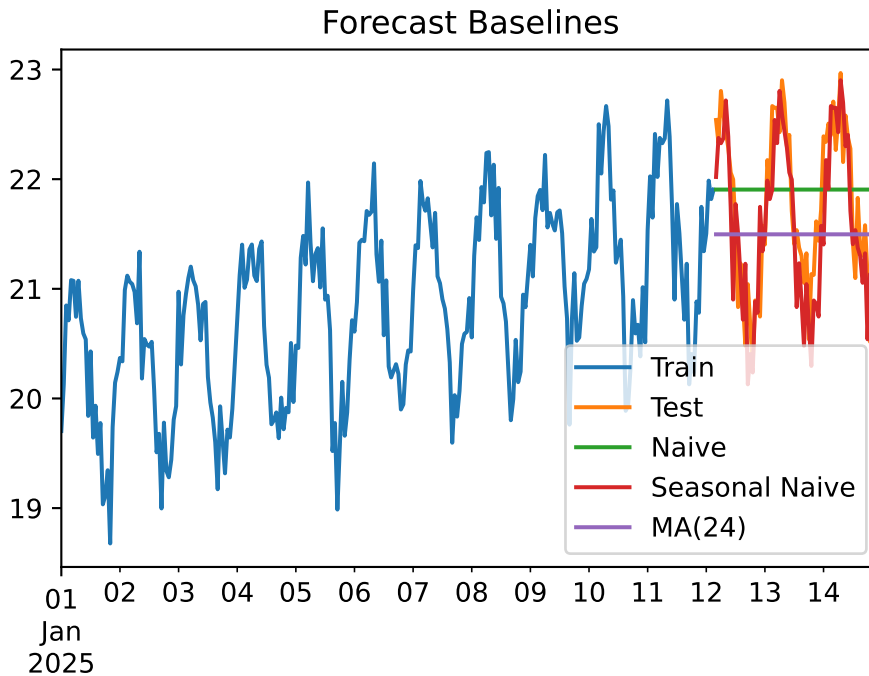
```
ma_value = train.iloc[-24:].mean()
ma_forecast = test.copy()
ma_forecast[:] = ma_value
```

16.11 NumPy

```
ma_value_np = np.mean(train_np[-24:])
ma_forecast_np = np.full_like(test_np, ma_value_np)
```

16.12 8.6 Visualizing Comparison

```
plt.figure()
train.plot(label="Train")
test.plot(label="Test")
naive.plot(label="Naive")
seasonal_naive.plot(label="Seasonal Naive")
ma_forecast.plot(label="MA(24)")
plt.legend()
plt.title("Forecast Baselines")
plt.show()
```



16.13 8.7 Error Metrics

We use the Mean Absolute Error (MAE):

$$MAE = \frac{1}{n} \sum |y_{true} - y_{pred}|$$

16.14 Pandas

```
def mae(y_true, y_pred):
    return np.mean(np.abs(y_true - y_pred))

print("MAE Naive:", mae(test, naive))
print("MAE Seasonal Naive:", mae(test, seasonal_naive))
print("MAE MA(24):", mae(test, ma_forecast))
```

MAE Naive: 0.6298837368696829
MAE Seasonal Naive: 0.3320412574311427
MAE MA(24): 0.6228787570177815

16.15 NumPy

```
def mae_np(y_true, y_pred):  
    return np.mean(np.abs(y_true - y_pred))  
  
print("MAE Naive:", mae_np(test_np, naive_np))  
print("MAE Seasonal:", mae_np(test_np, seasonal_naive_np))  
print("MAE MA(24):", mae_np(test_np, ma_forecast_np))
```

MAE Naive: 0.6298837368696829
MAE Seasonal: 0.3320412574311427
MAE MA(24): 0.6228787570177815

16.16 8.8 Interpretation

Typical observations:

- With a strong seasonal pattern, Seasonal Naive often performs best.
 - With a weak pattern, Naive can be surprisingly strong.
 - Moving Average smooths the data but reacts slowly to trend changes.
-

Methodological Note

Comparing to a baseline is not a formality, but a quality check.

16.17 8.9 Mini Exercises

1. Increase the strength of the seasonality.
 - Which baseline performs best?
2. Remove the seasonality entirely.
 - Which baseline performs better?
3. Add a strong trend.
 - How do the models respond?

17 9 Learning Objectives Check

18 9 Learning Objectives Check

Use this checklist for self-assessment.

18.1 9.1 Technical Competencies

I can ...

- ☐ methodically classify a time series (measurement process, sampling rate).
 - ☐ validate a time series (sorting, duplicates, missing values).
 - ☐ apply resampling appropriately (aggregation vs. interpolation).
 - ☐ distinguish between Rolling Mean and EWMA and choose appropriately.
 - ☐ conceptually separate trend and seasonality.
 - ☐ generate and interpret lag features.
 - ☐ interpret autocorrelation.
 - ☐ define anomalies globally and locally.
 - ☐ implement baseline forecasts.
 - ☐ evaluate forecast models using MAE.
-

18.2 9.2 Comprehension Questions

1. Why is the sampling rate crucial for interpreting seasonality?
 2. When is interpolation problematic?
 3. Why can a trend artificially increase autocorrelation?
 4. Why are baselines methodologically important?
 5. How does a structural break differ from an outlier?
-

 Note

If several points are unclear, review the corresponding chapters and work on the exercises.

19 Exercise 10: Time Series Workflow

20 Exercise 10: Time Series Workflow

This exercise combines several methods from the module.

20.1 Task 1: Generate Your Own Time Series

Create a time series with:

- 10 days, hourly frequency
- Trend
- Daily seasonality
- Noise
- At least 3 anomalies

Visualize the time series.

20.2 Task 2: Validation

Intentionally introduce errors:

- Duplicates
- Missing values
- Unsorted timestamps

Implement a proper validation.

20.3 Task 3: Resampling

- Aggregate to daily averages.
 - Compare original and aggregated series.
 - Interpret the information loss.
-

20.4 Task 4: Smoothing

- Test rolling windows of 6H, 24H, 72H.
 - What structure becomes visible in each case?
-

20.5 Task 5: Anomalies

- Implement an EWMA baseline.
 - Detect anomalies.
 - Vary the threshold.
-

20.6 Task 6: Forecast Baselines

- Train-test split (80/20)
- Naive
- Seasonal Naive (24H)
- MA(24)

Compare the MAE values and interpret the results.

i Submission Format (optional)

Plots + brief interpretation (bullet points).

21 11 Exam Questions

22 11 Exam Questions

22.1 Conceptual Questions

1. Define a time series.
 2. What is the difference between aggregation and interpolation?
 3. Explain rolling mean and EWMA conceptually.
 4. What is a lag feature?
 5. What does autocorrelation measure?
 6. Why can a trend affect autocorrelation?
 7. Explain the difference between global and local anomaly detection.
 8. What is a forecast baseline?
 9. Why are baselines methodologically necessary?
 10. What does MAE measure?
-

22.2 Application Questions

1. You receive machine data with a strong weekly pattern.
 - What steps would you take?
 2. A time series suddenly shows permanently higher values.
 - Outlier or structural break? Justify your answer.
 3. Two time series need to be compared, one measured every minute, the other hourly.
 - What steps are necessary?
-

Exam Preparation Note

Always justify your answers methodologically. Pure definitions are usually not sufficient.